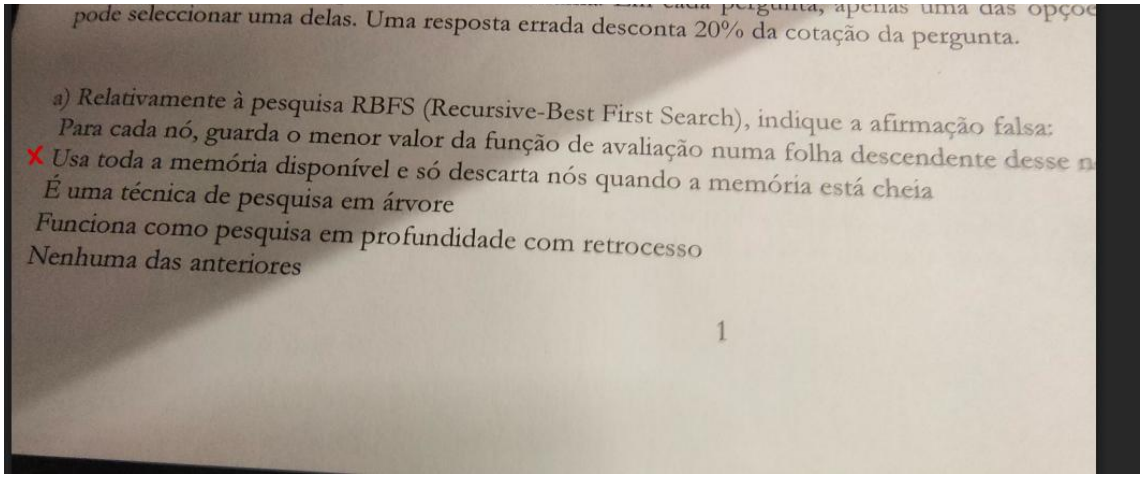
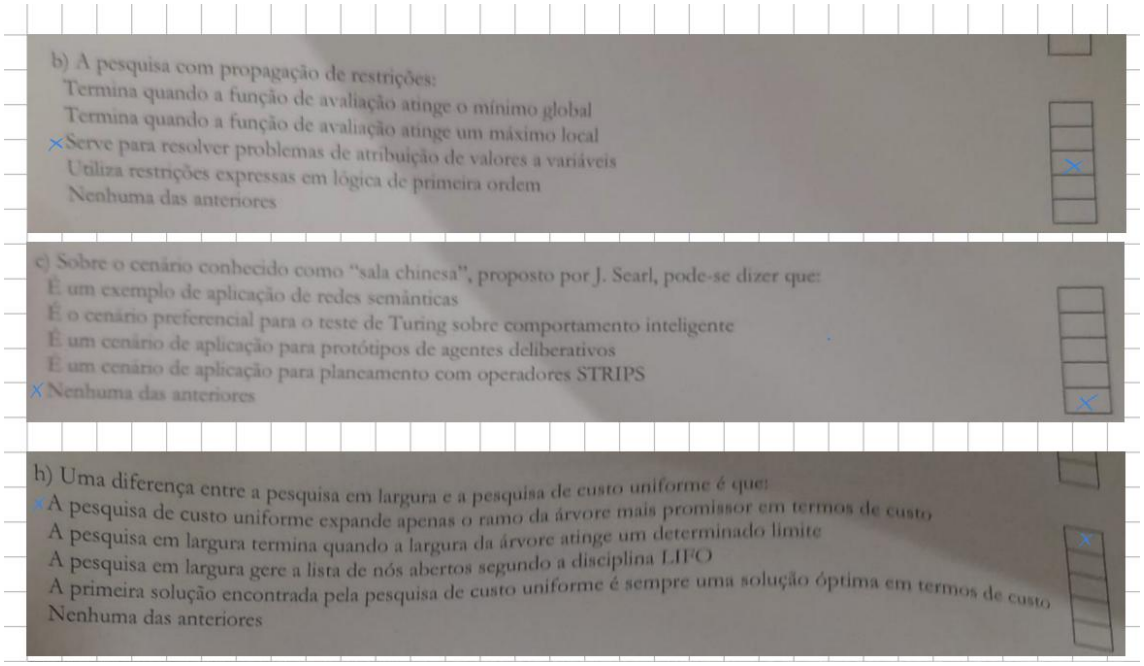
EM



pode seleccionar uma delas. Uma resposta errada desconta 20% da cotação da pergunta.

a) Relativamente à pesquisa RBFS (Recursive-Best First Search), indique a afirmação falsa:
Para cada nó, guarda o menor valor da função de avaliação numa folha descendente desse n
X Usa toda a memória disponível e só descarta nós quando a memória está cheia
É uma técnica de pesquisa em árvore
Funciona como pesquisa em profundidade com retrocesso
Nenhuma das anteriores

1



b) A pesquisa com propagação de restrições:
Termina quando a função de avaliação atinge o mínimo global
Termina quando a função de avaliação atinge um máximo local
X Serve para resolver problemas de atribuição de valores a variáveis
Utiliza restrições expressas em lógica de primeira ordem
Nenhuma das anteriores

c) Sobre o cenário conhecido como "sala chinesa", proposto por J. Searl, pode-se dizer que:
É um exemplo de aplicação de redes semânticas
É o cenário preferencial para o teste de Turing sobre comportamento inteligente
É um cenário de aplicação para protótipos de agentes deliberativos
É um cenário de aplicação para planeamento com operadores STRIPS
X Nenhuma das anteriores

h) Uma diferença entre a pesquisa em largura e a pesquisa de custo uniforme é que:
X A pesquisa de custo uniforme expande apenas o ramo da árvore mais promissor em termos de custo
A pesquisa em largura termina quando a largura da árvore atinge um determinado limite
A pesquisa em largura gere a lista de nós abertos segundo a disciplina LIFO
A primeira solução encontrada pela pesquisa de custo uniforme é sempre uma solução óptima em termos de custo
Nenhuma das anteriores

e) Uma diferença entre a pesquisa em largura e a pesquisa de custo uniforme é que:

- A pesquisa de custo uniforme expande apenas o ramo da árvore mais promissor
- A pesquisa de custo uniforme trabalha com um grafo de restrições
- A pesquisa em largura expande sempre o nó com menor custo
- A pesquisa em largura gere a lista nós abertos segundo a disciplina FIFO
- Nenhuma das anteriores

f) O algoritmo de pesquisa em grafo (*graph-search*) difere do algoritmo de pesquisa em árvore (*tree-search*) em q

- A pesquisa em grafo utiliza-se transições sempre na forma de operadores STRIPS
- A pesquisa em grafo trabalha com um grafo de restrições
- A pesquisa em grafo não cria nós com estados repetidos no caminho de cada nó até à raiz
- A pesquisa em grafo é completa e óptima
- Nenhuma das anteriores

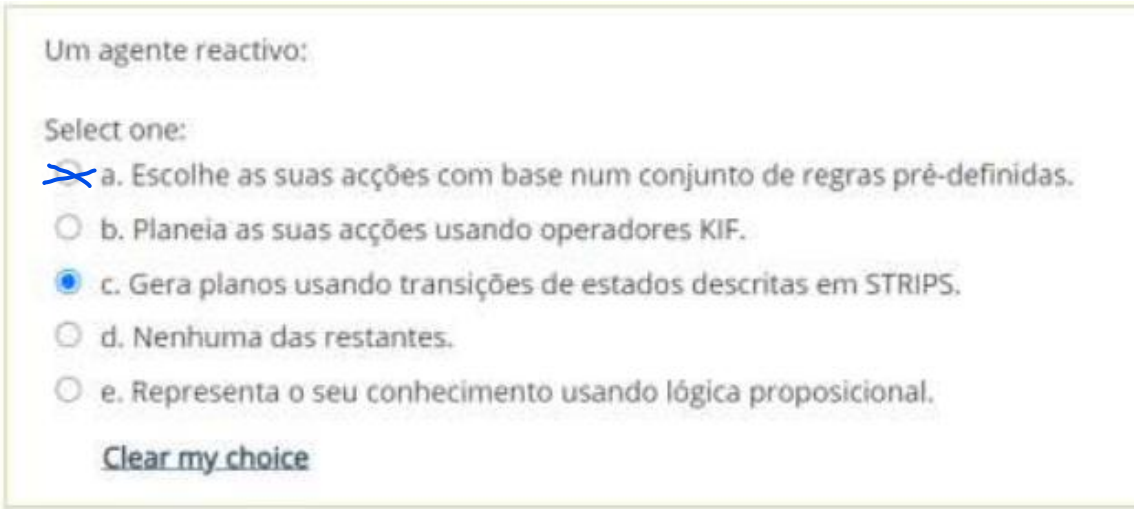
Quanto à arquitectura das três camadas, pode dizer-se que:

Select one:

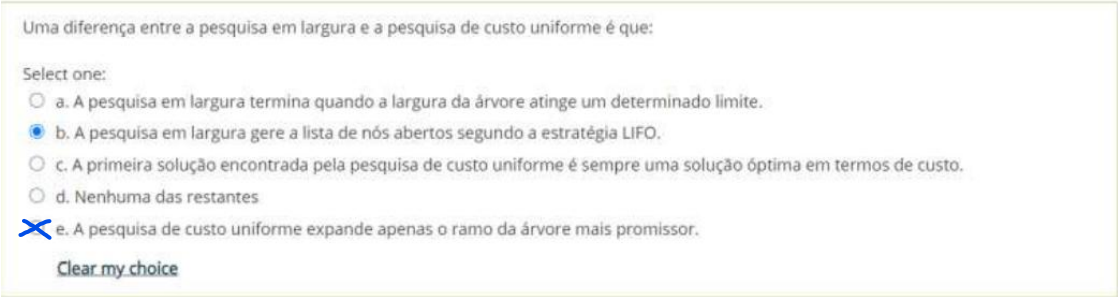
- a. Nenhuma das restantes.
- b. A camada central contém o conjunto das acções de nível de abstracção intermédio.
- c. Cada camada recebe dados sensoriais e produz um resultado que será combinado com os resultados das outras camadas.
- d. Não tem capacidade para reagir a situações de emergência.
- e. A camada central centra-se nos processos de memória e representação do conhecimento.

Clear my choice

Res posta e a B



Resposta e a A



Resposta e a C

Función 1

python Copy Download

```python
def f(x, y, z):
    if y == []:
        return [x]
    if z(x, y[0]):
        return [x] + y
    return [y[0]] + f(x, y[1:], z)
```

Explicación: Inserta `x` en la lista `y` manteniendo un orden definido por la función `z`. Si `z(x, y[0])` es True, inserta `x` al principio, sino recursivamente busca la posición correcta en el resto de la lista.

Función 2

python Copy Download

```python
def h(x, y, z):
    if x == []:
        return y[:]
    if y == []:
        return x[:]
    if z(x[0]) < z(y[0]):
        return [x[0]] + h(x[1:], y, z)
    return [y[0]] + h(x, y[1:], z)
```

Explicación: Combina dos listas ordenadas `x` e `y` usando merge sort. Compara elementos usando `z(elemento)` como clave de ordenamiento y fusiona recursivamente.

Función 3

python Copy Download

```python
def f(x,y):
    if x == [] or y==[]:
        return []
    return [(x[0],y[0])] + f(x[1:],y[1:])
```

Explicación: Crea una lista de pares (tuplas) combinando elementos correspondientes de `x` e `y`. Es equivalente a `zip(x, y)` en Python.

Función 5

python Copy Download

```python
def f(x,y):
    return [z for z in x if y(z)] != []
```

Explicación: Verifica si existe al menos un elemento en `x` que cumple con el predicado `y`. Retorna True si la lista filtrada no está vacía.

Función 6

python Copy Download

```python
def g(x,y):
    if y == []:
        return (0, [])
    (z, t) = g(x, y[1:])
    if y[0] == x:
        return (z + 1, t)
    return(z, y[1:] + t)
```

Explicación: Cuenta las ocurrencias de `x` en la lista `y` y retorna una tupla con: (cantidad de ocurrencias, lista original sin las ocurrencias de `x`).

Función 7

python Copy Download

```python
def f(x, y):
    if x == []:
        return None
    z = f(x[1:], y)
    if z == None or y(x[0], z):
        return x[0]
    return z
```

Explicación: Encuentra el elemento óptimo en `x` según la función de comparación `y`. Si `y(a, b)` es True, `a` es "mejor" que `b`. Retorna el elemento óptimo de la lista.

Función 8 (Incompleta)

python Copy Download

```python
def g(x, y):
    if x == [] and y == []:
        return []
    if x != [] and y != []:
        z = g(x[1:], y[1:])
        if z != None:
            return [(x[0], y[0])] + z
```

Explicación: Función incompleta que intenta hacer un zip como la función 3, pero le falta manejar casos donde las listas tienen longitudes diferentes.

```python
def get_ancestors(bn, event):
    ancestors = []
    for i in bn:
        if i[0] == event:
            for j in i[1]:
                ancestors.append(j[0])
                ancestors.extend(get_ancestors(bn, j[0]))
    return list(set(ancestors))
```

```python
'''lst é uma lista que contem listas.
    Listas com apenas um elemento sao proposiçoes, i.e factos.
    Listas com Mais de um elemento sao implicações, e.g [c, a, b] quer dizer que a E b implica c.
    Logo, provar([['a'],['b'], ['c', 'a', 'b']], 'd') -> True'''

def provar(conhecimento, prop):
    v = [f[0] for f in conhecimento if len(f) == 1]
    for i in range(len(conhecimento)):
        for f in conhecimento:
            if len(f) > 1 and all(p in v for p in f[1:]) and f[0] not in v:
                v.append(f[0])
    return prop in v


print(provar([['a'], ['b'], ['c', 'a', 'b']], 'd'))
```

```python
def novos_abertos(x,y):
    if x == []:
        return y

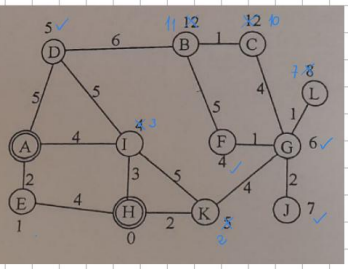
    if x[0] in y:
        return f(x[1:],y)

    return f(x[1:],y) + [x[0]]
```

A função f, dadas duas listas, cria uma nova lista, de tuplos, com a concatenação dos pares de elementos das listas dadas, para cada index, de forma recursiva, até uma das listas acabar. A lista resultante, tem o comprimento da maior lista

4. O grafo apresentado ao lado representa um espaço de estados, sendo **A** e **H** os estados que satisfazem o objectivo. Os custos das transições estão anotados junto às ligações do grafo e os valores da heurística estão anotados junto a cada nó.

a) A heurística apresentada na figura é admissível? Justifique a sua resposta e, em caso negativo, indique possíveis alterações que a tornem admissível. [Pode fazer directamente na figura.]



Para a heurística ser admissível, não pode sobrestimar o custo real de chegar a uma solução a partir do nó, o que não se verifica nos nós B, C, I, K e L, pois tem um custo superior ao custo real.

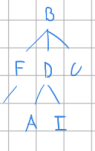
A função heurística $h(n)$ diz-se *admissível* se nunca sobrestima o custo real de chegar a uma solução a partir de n.

b) Tomando o estado B como estado inicial

sobrestima o custo real de chegar a uma solução a partir de *n*.

b) Tomando o estado B como estado inicial, apresente a árvore de pesquisa gerada quando se realiza uma pesquisa gulosa. Esta pesquisa é feita sem repetição de estados no caminho de qualquer nó até à raiz da árvore. Numere os nós pela ordem em que são acrescentados à árvore e, caso use função de avaliação, anote também o valor da função de avaliação em cada nó. Em situações de empate na ordenação dos nós, deve desempatar com base na ordem alfabética dos respectivos estados. [Nota: caso seja necessário usar a heurística, deve usar os valores originais.]

1º - B (0)
2º - F (4)
3º - D (5)
4º - C (12)
5º - G (6)
6º - A (0)
7º - I (4)



Em primeiro lugar é adicionado à arvore o nó B, que expandido dá origem aos nós F, D e C, de seguida, destes nós é escolhido o nó F para ser expandido, pois é o que tem a menor heurística dos 3, esta expansão dá origem ao nó G, tendo os nós D, C e G, é expandido o nó D, pois é o que tem a menor heurística ...

* O nó solução só é encontrado quando este é expandido, e não quando é resultado da expansão do seu nó pai.

Pesquisa gulosa

- Ignora custo acumulado $g(n)$
- $f(n) = h(n)$

– Dado que o custo acumulado é ignorado, não é verdadeiramente um caso particular da pesquisa A*
– Tem um comportamento que se aproxima da pesquisa em profundidade
– Ao ignorar o custo acumulado, facilmente deixa escapar a solução óptima

A pesquisa gulosa escolhe sempre expandir o nó com menor heurística

c) Qual o custo da solução encontrada? A solução encontrada é óptima? Justifique.

Optimalidade – a primeira solução que a estratégia de pesquisa consegue encontrar é a melhor solução.

B → D → A = 6 + 5 = 11

A solução é ótima, porque a primeira solução encontrada, é a de menor custo

5. Na margem esquerda do rio Vouga estão N pessoas, que pretendem atravessar o rio para a margem direita, e um barco. Este pode transportar no máximo duas pessoas, uma das quais no lugar do condutor e outra como simples passageiro. As pessoas podem embarcar, conduzir o barco e desembarcar.

a) Proponha alguns predicados que considere adequados para representar as condições que se podem verificar em diferentes estados.

Esquerda (p) Esquerda (b) p in {pessoas}
Direita (p) Direita (b) b in {barco}

Condutor (p)
Passageiro (p)

b) Usando os predicados que propôs na alínea anterior, especifique um conjunto de operadores STRIPS para representar as acções que podem ser realizadas no atravessamento do rio.

5) STRIPS

barco max 2 pessoas, Esquerda → direita,

Um condutor, passageiro, embarcar, Atravessar, desembarcar

a) ~~[illegible]~~ Barco(Pessoa) passageiro

Margem Esq (Pessoa)

Margem dir (Pessoa)

Barco Esq ()

Barco Dir ()

Condutor (Pessoa)

Barco Vazio()

Barco cheio ()

3) 1 operador: Embarcar

Pre_condição:

Barco Esq ()

Margem Esq (Pessoa)

Delete:

Barco Vazio()

add list:

Barco (Pessoa)

Condutor (Pessoa)

2 op: Atravessar

Pre:

Barco (condutor)

Baro Esq ou BarcoDir

Delete:

Barco Esq

ou

Baro dir

add:

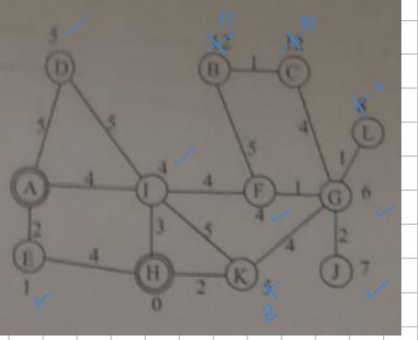
Barco dir ou esq

3 op

Pr

2)

5. O grafo apresentado ao lado representa um espaço de estados, sendo **A** e **H** os estados que satisfazem o objectivo. Os custos das transições estão anotados junto às ligações do grafo e os valores da heurística estão anotados junto a cada nó.

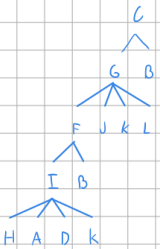


a) A heurística apresentada na figura é admissível? Justifique a sua resposta e, em caso negativo, indique possíveis alterações que a tornem admissível. [Pode fazer directamente na figura.]

A heurística apresentada na figura não é admissível, pois nos nós B, C, K e L está a sobrestimar o custo real dos nós para o objetivo.

b) Tomando o estado **C** como estado inicial, apresente a árvore de pesquisa gerada quando se realiza uma pesquisa A*. Esta pesquisa é feita sem repetição de estados no caminho de qualquer nó até à raiz da árvore. Numere os nós pela ordem em que são acrescentados à árvore e, caso use função de avaliação, anote também o valor da função de avaliação em cada nó. Em situações de empate na ordenação dos nós, deve desempatar com base na ordem alfabética dos respectivos estados. [Nota: caso seja necessário usar a heurística, deve usar os valores originais.]

1º - C
2º - G g(n) = 4 h(n) = 6 → 10
3º - B g(n) = 1 h(n) = 12 → 13
4º - F g(n) = 5 h(n) = 4 → 9
5º - J g(n) = 6 h(n) = 7 → 13
6º - K g(n) = 8 h(n) = 5 → 13
7º - L g(n) = 5 h(n) = 8 → 13
8º - I g(n) = 9 h(n) = 4 → 13
9º - B g(n) = 6 h(n) = 11 → 17
10º - H g(n) = 12 h(n) = 0 → 12
11º - A g(n) = 13 h(n) = 0 → 13
12º D g(n) = 14 h(n) = 5 → 19
13º - K g(n) = 14 h(n) = 5 → 19



Em primeiro lugar é adicionado o nó C à árvore, que expandido dá origem aos nós B e G. Na pesquisa A* é escolhido o nó com a função de custo total mínima (custo até à raiz + heurística), logo o próximo nó a ser expandido é o G...

Pesquisa A*

- Escolhe-se o nó em que a função de custo total $f(n)=g(n)+h(n)$ tem o menor valor
  - $g(n)$ = custo desde o nó inicial até ao nó n
  - $h(n)$ = custo estimado desde o nó n até à solução [heurística]
- A função heurística $h(n)$ diz-se *admissível* se nunca sobrestima o custo real de chegar a uma solução a partir de n.
- Se for possível garantir que $h(n)$ é admissível, então a pesquisa A* encontra sempre (um)a solução óptima.
- A pesquisa A* é também completa.

c) Calcule o factor de ramificação efectivo da árvore gerada.

c) Calcule o factor de ramificação efectivo da árvore gerada.

$N = \frac{B^{d+1} - 1}{B - 1} \Leftrightarrow \frac{B - 1}{B - 1}$

$\Leftrightarrow \frac{B - 1}{B - 1}$

$\Leftrightarrow (B-1) = B - 1$

$\Leftrightarrow - = B$

$\Leftrightarrow B - 11B + 10 = 0$

$\Leftrightarrow B =$

Avaliação da pesquisa em árvore - factores de ramificação

- Seja:
  - N – número de nós da árvore de pesquisa no momento em que se encontra a solução
  - X – Número de nós expandidos (não terminais)
  - d – comprimento do caminho na árvore correspondente à solução
- *Ramificação média* – número médio de filhos por nó expandido:

$$RM = \frac{N-1}{X}$$

Nota: a ramificação média é um indicador da dificuldade do problema.

- *Factor de ramificação efectivo* – número de filhos por nó, B, numa árvore com ramificação constante e com profundidade constante *d*. Portanto: $1 + B + B^2 + ... + B^d = N$ ou seja: $\frac{B^{d+1} - 1}{B - 1} = N$ (resolve-se por métodos numéricos).
  - O factor de ramificação efectiva é um indicador da eficiência da técnica de pesquisa utilizada.

3. Apesar de a pesquisa A* ter um comportamento muito bom para muitos problemas reais, esta técnica não deixa de ter um comportamento exponencial. Assim, em problemas especialmente complexos, a pesquisa A* pode não conseguir resolvê-los devido a limitações de memória. Conhece técnicas de pesquisa desenhadas para lidar com esta limitação da pesquisa A*? Se sim, descreva as suas principais características.

Para lidar com esta limitação, temos a pesquisa IDA, que é semelhante à pesquisa A*, no entanto, tem um limite para a função f(n), ou seja, qualquer nó com f(n) > f máx, não é expandido. Existe também a pesquisa recursiva melhor - primeiro que faz cada nó, o programa não guarda o valor da função avaliação, mas sim, o menor valor de f(x), sendo x uma folha descendente do nó n. Também temos a pesquisa SMA, que quando a memória chega ao limite, remove o nó com maior custo, e atualiza nos nós ascendentes o custo do melhor nó esquecido, para assim, nó voltar a gerar esse nó, quando o custo do nomó for inferior ao dos restantes. Com as características acima referidas, estes tipos de pesquisa conseguem solucionar os problemas de limitação de memória da pesquisa A*.

SMA*

- A* com memória limitada simplificado (*simplified memory-bounded A**)
- Usa a memória disponível
  - Contraste com IDA* e RBFS: estes foram desenhados para poupar memória, independentemente de ela existir de sobra ou não
- Quando a memória chega ao limite, esquece (remove) o nó *n* com maior custo $f(n)=g(n)+h(n)$, actualizando em cada um dos nós ascendentes o "custo do melhor nó esquecido"
- Só volta a gerar o nó *n* quando o custo do melhor nó esquecido registado no antecessor de *n* for inferior aos custos dos restantes nós
- Em cada iteração, é gerado apenas um nó sucessor
  - Existindo já um ou mais filhos de um nó, apenas se gera ainda outro se o custo do nó pai for menor do que qualquer dos custos dos filhos
  - Quando se gerou todos os filhos de um nó, o custo do nó pai é actualizado como no RBFS

RBFS

- Pesquisa recursiva melhor-primeiro (*Recursive Best-First Search*)
- Para cada nó *n*, o algoritmo não guarda o valor da função de avaliação $f(n)$, mas sim o menor valor $f(x)$, sendo *x* uma folha descendente do nó *n*
  - Sempre que um nó é expandido, os custos armazenados nos ascendentes são actualizados
- Funciona como pesquisa em profundidade com retrocesso
  - Quando a folha *m* com menor custo $f(m)$ não é filha do último nó expandido *n*, então o algoritmo retrocede até ao ascendente comum de *m* e *n*

IDA*

- Semelhante à pesquisa em profundidade com aprofundamento iterativo
- A limitação à profundidade é estabelecida indirectamente através de um limite na função de avaliação $f(n) = g(n)+h(n) \leq f_{max}$
  - Ou seja: Qualquer nó *n* com $f(n) > f_{max}$ não será expandido
- Passos do algoritmo:
  1. $f_{max} = f(\text{raiz})$
  2. Executar pesquisa em profundidade com limite f_{max}
  3. Se encontrou solução, retornar solução encontrada
  4. $f_{max} \leftarrow$ menor $f(n)$ que tenha sido superior a f_{max} na última execução do A*
  5. Voltar ao passo 2

2. Identifique e caracterize os principais tipos de agentes inteligentes bem como pelo menos uma arquitectura hierárquica para agentes mais complexos.

Agente reativo simples – realiza ações com base na perceção do estado do ambiente atual

Agente reativo com estado interno – realiza ações com base na perceção do estado do ambiente atual e no histórico de perceções observadas sobre o estados de ambientes anteriores

Agente deliberativo orientado por objetivos – como a perceção atual e o histórico de perceções sobre o estado do ambiente pode não ser suficiente, este agente, realiza ações, com base em objetivos, que descrevem situações a atingir

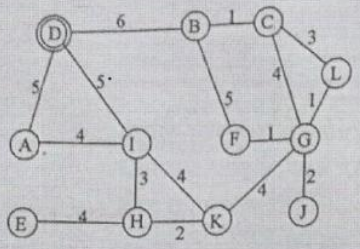
Agente deliberativo orientado por função de utilidade – quando o histórico e a perceção atual sobre o estado do ambiente, e os objetivos, não são suficientes, para determinar uma ação existem funções de utilidade, que comparam os diferentes estados e avaliam qual a melhor ação a realizar

Arquitetura de Subsunção – Organiza o seu comportamento em camadas hierárquicas, os inferiores, são para ações rápidas e os superiores, são para lidar com objetivos complexos e planeamento.

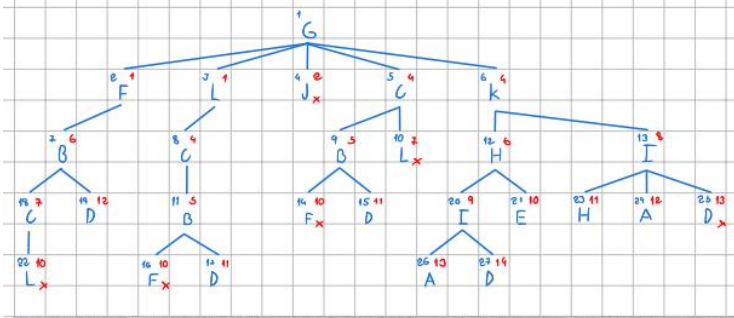
```
def h(x,y,z):
  if x==[]:
    return y[:]
  if y==[]:
    return x[:]
  if z(x[0]) < z(y[0]):
    return [x[0]]+h(x[1:],y,z)
  return [y[0]]+h(x,y[1:],z)
```

A função h, recursivamente, dadas duas listas ordenadas e uma função lambda, faz a concatenação das duas listas, de forma ordenada

3. O grafo a seguir apresentado representa um espaço de estados num problema de pesquisa, sendo D o estado objectivo (solução). Os custos das transições estão anotados junto às ligações do grafo.



a) Tomando o estado G como estado inicial, apresente a árvore de pesquisa gerada quando se realiza uma pesquisa de custo uniforme. Esta pesquisa é feita sem repetição de estados no caminho de qualquer nó até à raiz da árvore. Numere os nós pela ordem em que são acrescentados à árvore e anote também o valor da função de avaliação em cada nó. Em caso de empate nos valores da função de avaliação em dois ou mais nós, utilize a desempate com base na ordem alfabética dos respectivos estados.



Ordem de Adição
Custo

b) Calcule o factor de ramificação médio da árvore gerada.

$R_m = \frac{N-1}{x} = \frac{27-1}{13} = \frac{26}{13} = 2$ $\quad N = 27$ $\quad x = 3$